

Anexo 07 — Evaluacion de la policy de cabeza

Archivo: Scripts/eval_ppo.py

Para que sirvio:

Corre N episodios de seguimiento, reporta estadisticas (angulo, foco) y opcionalmente graba un video con overlay.

```
1 | """Evaluacion de una policy PP0 entrenada. Genera estadisticas y opcionalmente video.
2 |
3 | Uso:
4 |     python eval_ppo.py runs/pp0_dum_v8_tuned/final.zip
5 |     python eval_ppo.py runs/pp0_dum_v8_tuned/final.zip --video out.mp4 --episodes 3
6 |     python eval_ppo.py runs/pp0_dum_v8_tuned/final.zip --video # autogenera eval_v8.mp4
7 | """
8 | import argparse
9 | import re
10 | import sys
11 | from pathlib import Path
12 |
13 | sys.path.insert(0, str(Path(__file__).resolve().parent))
14 |
15 | import numpy as np
16 | import mujoco
17 | from stable_baselines3 import PPO
18 | from rl_env import DUMHeadTrackingEnv
19 |
20 |
21 | def evaluate(model, env, n_episodes=10, deterministic=True):
22 |     """Corre n_episodes y devuelve estadisticas agregadas."""
23 |     rewards, lengths = [], []
24 |     final_thetas = []
25 |     focus_fires = 0
26 |     for ep in range(n_episodes):
27 |         obs, _ = env.reset(seed=ep)
28 |         done, trunc = False, False
29 |         ep_rew = 0.0
30 |         ep_len = 0
31 |         thetas = []
32 |         focused = False
33 |         while not (done or trunc):
34 |             action, _ = model.predict(obs, deterministic=deterministic)
35 |             obs, r, done, trunc, info = env.step(action)
36 |             ep_rew += r
37 |             ep_len += 1
38 |             thetas.append(info["theta_deg"])
39 |             if info["focus_triggered"]:
40 |                 focused = True
41 |             rewards.append(ep_rew)
42 |             lengths.append(ep_len)
43 |             final_thetas.append(thetas[-1])
44 |             if focused:
45 |                 focus_fires += 1
46 |             print(f"ep {ep:2d}: rew={ep_rew:+7.2f} len={ep_len} "
47 |                   f"theta_min={min(thetas):.2f} deg theta_final={thetas[-1]:.2f} deg "
48 |                   f"focus={'YES' if focused else 'no'}")
49 |         print()
50 |         print(f"Sobre {n_episodes} episodios:")
51 |         print(f"  ep_rew_mean = {np.mean(rewards):+.2f} ± {np.std(rewards):.2f}")
52 |         print(f"  theta_final mean = {np.mean(final_thetas):.2f} deg ± {np.std(final_thetas):.2f}")
53 |         print(f"  focus_triggered en {focus_fires}/{n_episodes} episodios")
54 |         return dict(rewards=rewards, final_thetas=final_thetas, focus_fires=focus_fires)
55 |
56 |
57 | def record_video(model, env, out_path, n_episodes=2, fps=50, model_path=""):
58 |     """Renderiza n_episodios con overlay informativo y los guarda en un mp4.
59 |
60 |     El control corre a 50 Hz (timestep=0.005 * frame_skip=4).
61 |     Si se guarda a 50 fps, el video es 1x real-time (asumiendo que el reproductor honra esa fps).
62 |     """
63 |     import imageio
64 |     from PIL import Image, ImageDraw, ImageFont
65 |
66 |     # Tamaño limitado al offscreen framebuffer (default 480x480 si no se especifica en el XML
67 |     # vía <visual><global offwidth=... offheight=.../></visual>)
68 |     renderer = mujoco.Renderer(env.model, height=480, width=480)
69 |
70 |     try:
71 |         font = ImageFont.truetype("arial.ttf", 13)
72 |         font_small = ImageFont.truetype("arial.ttf", 11)
73 |     except Exception:
74 |         font = ImageFont.load_default()
```

```

75 |         font_small = ImageFont.load_default()
76 |
77 |     model_name = Path(model_path).name if model_path else "unknown"
78 |     try:
79 |         total_steps = int(model.num_timesteps)
80 |     except Exception:
81 |         total_steps = "?"
82 |
83 |     frames = []
84 |     for ep in range(n_episodes):
85 |         obs, _ = env.reset(seed=ep)
86 |         done, trunc = False, False
87 |         step_idx = 0
88 |         ep_reward = 0.0
89 |         while not (done or trunc):
90 |             action, _ = model.predict(obs, deterministic=True)
91 |             obs, r, done, trunc, info = env.step(action)
92 |             ep_reward += r
93 |             step_idx += 1
94 |
95 |             renderer.update_scene(env.data, camera=-1)
96 |             frame = renderer.render()
97 |             img = Image.fromarray(frame)
98 |             draw = ImageDraw.Draw(img)
99 |
100 |             # Header (sup izq) - info del modelo
101 |             header_lines = [
102 |                 f"model: {model_name}",
103 |                 f"trained: {total_steps:,} steps",
104 |                 f"speed: 1x real-time @ {fps} fps",
105 |             ]
106 |             y = 6
107 |             for line in header_lines:
108 |                 draw.text((6, y), line, fill=(230, 230, 230), font=font_small)
109 |                 y += 13
110 |
111 |             # Estado del episodio (sup der)
112 |             focus_str = "YES" if info["focus_triggered"] else "no"
113 |             mode_str = info.get("target_mode", "?")
114 |             ep_lines = [
115 |                 f"ep {ep} step {step_idx:3d}/500",
116 |                 f"target: {mode_str}",
117 |                 f"theta = {info['theta_deg']:5.1f} deg",
118 |                 f"ep_reward = {ep_reward:+7.2f}",
119 |                 f"focus: {focus_str}",
120 |             ]
121 |             y = 6
122 |             for line in ep_lines:
123 |                 tw = draw.textlength(line, font=font)
124 |                 draw.text((img.width - tw - 6, y), line, fill=(255, 240, 100), font=font)
125 |                 y += 14
126 |
127 |             frames.append(np.array(img))
128 |
129 |     imageio.mimsave(out_path, frames, fps=fps, quality=8)
130 |     print(f"Video guardado en {out_path} ({len(frames)} frames @ {fps} fps)")
131 |
132 |
133 | def main():
134 |     p = argparse.ArgumentParser()
135 |     p.add_argument("model_path", type=str)
136 |     p.add_argument("--episodes", type=int, default=15)
137 |     p.add_argument("--video", type=str, nargs='?', const='AUTO', default=None,
138 |                    help="Si se pasa '--video' sin valor, autogenera eval_v<N>.mp4 "
139 |                         "extrayendo el N del nombre del run. Si se pasa con ruta, la usa tal cual.")
140 |     p.add_argument("--stochastic", action="store_true")
141 |     args = p.parse_args()
142 |
143 |     # Resolver --video AUTO -> eval_v<N>.mp4 en la carpeta del run
144 |     if args.video == 'AUTO':
145 |         run_dir = Path(args.model_path).parent
146 |         m = re.search(r'v(\d+)', run_dir.name)
147 |         num = m.group(1) if m else "x"
148 |         args.video = str(run_dir / f"eval_v{num}.mp4")
149 |         print(f"Video auto-naming: {args.video}")
150 |
151 |     env = DUMHeadTrackingEnv()
152 |     model = PPO.load(args.model_path)
153 |     print(f"Loaded: {args.model_path}")
154 |     print()
155 |
156 |     evaluate(model, env, n_episodes=args.episodes, deterministic=not args.stochastic)
157 |
158 |     if args.video:
159 |         print()

```

```
160 |         record_video(model, env, args.video,
161 |                       n_episodes=min(3, args.episodes),
162 |                       model_path=args.model_path)
163 |
164 |
165 | if __name__ == "__main__":
166 |     main()
167 |
```